



**KUALA LUMPUR UNIVERSITY
OF SCIENCE AND TECHNOLOGY**

(formerly known as Infrastructure University Kuala Lumpur)

**ASSIGNMENT 1: Implementation and Analysis of 2D Graphics Primitives,
Transformations, and Viewing in OpenGL**

Name:	Mohamed Mohamed Hussein
Student ID:	251925403
Course:	CSC3031 – Computer Graphics
University:	Kuala Lumpur University of Science and Technology
Lecturer:	VALERIANO DASALLA
Semester	Oct/Nov 2025
Date:	January 5, 2026

TABLE OF CONTENTS

1.0 Introduction
2.0 Development Environment
3.0 Mathematical Theory
4.0 Task 1: Design and Implementation
4.1 Manual Plotting (Graph Paper)
4.2 The Scale Calculation
4.3 Graphic Primitives Used
5.0 Task 2: Interaction and Transformation
5.1 Translation (Movement)
5.2 Rotation (Spinning)
5.3 The Matrix Stack
6.0 Code Explanation
7.0 Challenges and Solutions
8.0 Conclusion
9.0 Appendix: Source Code

1.0 INTRODUCTION

The goal of this assignment is to create a 2D graphical object using the C++ programming language and the OpenGL library. The specific object we had to design is a mechanical gear. This gear must be made of simple shapes called "primitives," such as triangles and rectangles.

Computer graphics is an important part of Computer Science. It allows us to draw images on a screen using mathematics. In this project, I learned how to use the coordinate system to place points (vertices) on the screen. I also learned how to color these shapes and how to make them move.

The assignment has two main tasks. Task 1 is about drawing the static gear. I had to draw it on graph paper first to find the correct coordinates. Task 2 is about making the gear interactive. This means the user can move the gear up, down, left, or right using the keyboard. The user can also spin the gear using specific keys.

This report explains step-by-step how I designed the gear, the math I used, and how I wrote the code.

2.0 DEVELOPMENT ENVIRONMENT

To complete this assignment, I used specific software tools. It is important to set up the environment correctly so the graphics code can run.

2.1 Code::Blocks

I used Code::Blocks as my IDE (Integrated Development Environment). This is where I wrote my C++ code. It is good for students because it is free and easy to use.

2.2 GLUT (OpenGL Utility Toolkit)

OpenGL by itself is very complex. To make it easier, I used a library called GLUT. This library help me create a window, handle keyboard input, and draw simple shapes. Without GLUT, it would be very difficult to open a window on the computer.

2.3 Compiler

I used the MinGW compiler. This takes the human-readable C++ code and turns it into a program that the computer can run.

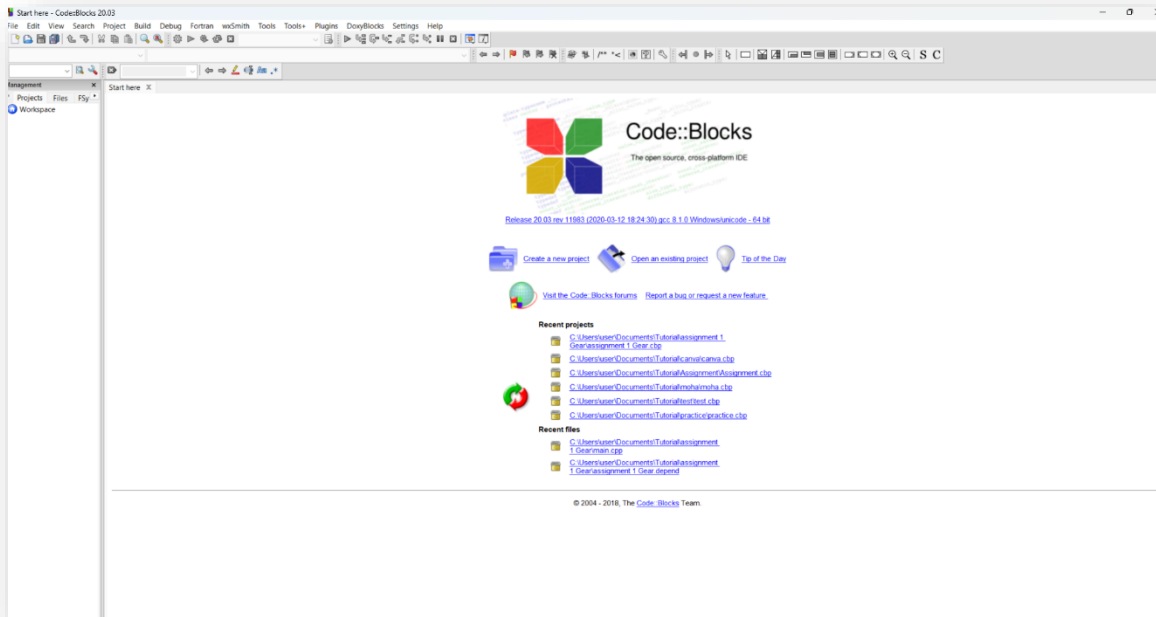


Figure 1: The Code::Blocks coding environment.

3.0 MATHEMATICAL THEORY

Before writing code, we must understand the math behind computer graphics. The computer screen is like a piece of graph paper.

3.1 Cartesian Coordinate System

OpenGL uses the Cartesian coordinate system. This means every point on the screen is defined by two numbers: X and Y.

- **X:** This is the horizontal position. Positive numbers are to the Right. Negative numbers are to the Left.
- **Y:** This is the vertical position. Positive numbers are Up. Negative numbers are Down.
- **Origin (0,0):** This is the exact center of the screen.

In my project, I placed the center of the gear at the Origin (0,0). This makes it easier to rotate the gear later.

3.2 Geometric Primitives

The computer cannot draw a "circle" directly. It can only draw straight lines and triangles. To make a circle, i draw many small triangles arranged in a fan shape. If i use enough triangles (for example, 100), it looks like a smooth circle. This is called "approximation."

4.0 TASK 1: DESIGN AND IMPLEMENTATION

This section explains how I built the gear. The assignment said the gear must be a "multi-part object." This means I could not just draw one shape. I had to combine three different parts to make the final gear.

4.1 Manual Plotting (Graph Paper)

First, I drew the gear by hand on graph paper. This was a very important step. It helped me visualize the sizes of the different parts.

I used colors to separate the parts:

- **Yellow:** For the center (The Hub).
- **Green:** For the main body ring.
- **Green with Black Outline:** For the teeth.

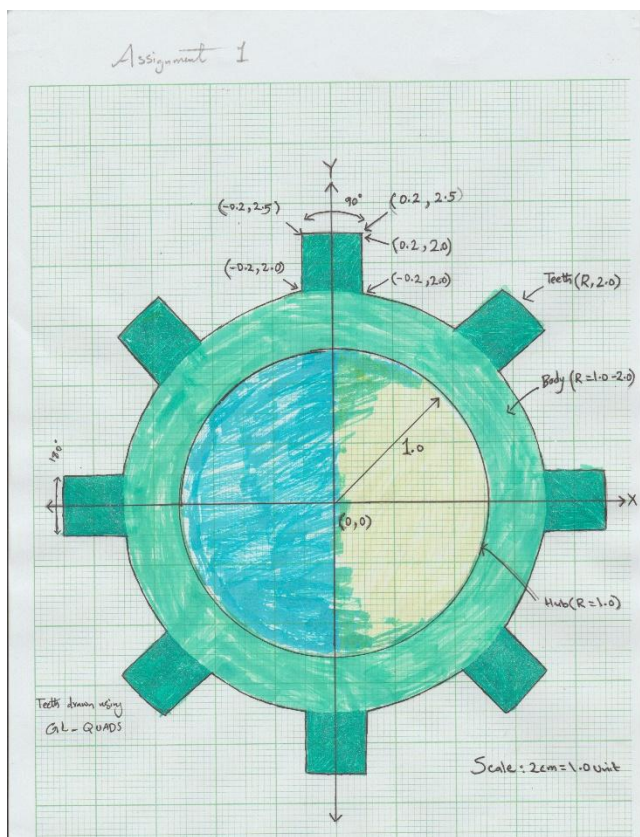


Figure 2: My manual design on graph paper.

4.2 The Scale Calculation

On my graph paper, I needed to define a "Scale." The assignment said the Inner Radius must be 1.0 Unit.

I measured my drawing with a ruler. The inner circle was 2.0 cm wide on the paper.

Therefore, my scale is:

2.0 cm = 1.0 Unit

Using this scale, I calculated the size of the teeth:

- The tooth tip was at 5.0 cm.
- Calculation: $5.0 / 2.0 = 2.5$.
- So, in my code, the tooth tip coordinate is **2.5**.

4.3 Graphic Primitives Used

I used three specific OpenGL primitives to build the gear.

Part 1: The Hub (GL_TRIANGLE_FAN)

The center of the gear is a solid circle. I used GL_TRIANGLE_FAN for this. This primitive starts at one center point (0,0) and connects to many points around the outside.

- **Coloring:** The assignment asked for "interpolated color." I set the center point to **Blue** and the outer points to **Yellow**. OpenGL blends these colors automatically.

Part 2: The Body (GL_QUAD_STRIP)

The body of the gear is a ring. A ring has an inner radius and an outer radius. The best primitive for this is GL_QUAD_STRIP. It creates a series of connected rectangles that follow a circular path.

- **Inner Radius:** 1.0
- **Outer Radius:** 2.0
- **Color:** Green

Part 3: The Teeth (GL_QUADS)

The teeth are rectangular shapes. I used GL_QUADS.

- **Color:** Green (to match the body).
- **Outline:** I also added a black outline using `GL_LINE_LOOP` so the teeth look distinct and sharp, just like my drawing.

5.0 TASK 2: INTERACTION AND TRANSFORMATION

Task 2 required the gear to move. In Computer Graphics, moving an object is called "Transformation." There are two types of transformations used in this project.

5.1 Translation (Movement)

Translation means moving an object from one place to another. We change the object's X position or Y position.

In my code, I used variables called `posX` and `posY`.

- When I press the **Up Arrow**, I increase `posY`.
- When I press the **Right Arrow**, I increase `posX`.

The function used is `glTranslatef(posX, posY, 0.0)`.

5.2 Rotation (Spinning)

Rotation means turning the object around a point. We want the gear to spin around its own center.

I used a variable called `angle`.

- When I press **'L'**, the angle increases (Counter-Clockwise).
- When I press **'R'**, the angle decreases (Clockwise).

The function used is `glRotatef(angle, 0.0, 0.0, 1.0)`. The 1.0 at the end means we are rotating around the Z-axis.

5.3 The Matrix Stack

The order of transformation is very important. If we rotate the object before we move it, it will spin around the corner of the screen. This is wrong.

We must Move (Translate) the object first, and then Rotate it.

My code follows this logic:

1. `glLoadIdentity();` (Reset everything)

2. `glTranslatef(...)`; (Move the gear to the correct location)
3. `glRotatef(...)`; (Spin the gear in place)
4. `drawGear()`; (Draw the picture)

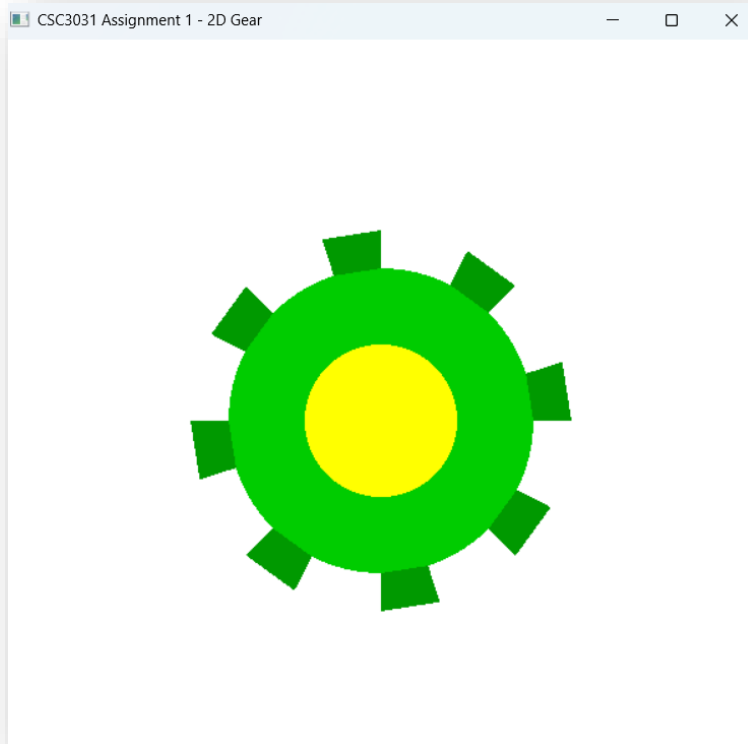


Figure 3: The gear after being moved using translation.

6.0 CODE EXPLANATION

This section explains the main parts of my C++ program.

The `main()` Function

This is where the program starts. It sets up the window size to 600 \times 600 pixels. It also tells GLUT to use "Double Buffering." Double buffering is very important for animation. It stops the screen from flickering when the gear spins.

The `drawGear()` Function

This is a custom function I wrote. It contains all the math to draw the gear. It has three sections inside it:

1. Draw the Yellow Hub.
2. Draw the Green Body Ring.
3. Draw the 8 Teeth using a for loop.

The display() Function

This function runs over and over again. It clears the screen to make it clean. Then, it applies the movement variables (posX, posY, angle). Finally, it calls drawGear() to put the image on the screen.

The keyboard() Function

This function listens for user input. If the user presses 'R' or 'L', this function updates the angle variable. It then tells the computer to redraw the screen immediately.

7.0 CHALLENGES AND SOLUTIONS

I faced a few problems while doing this assignment. Here is how I solved them.

Problem 1: The Teeth looked thin

At first, I drew the teeth and the body with the same green color. Because they were the same color, they blended together into a blob. The teeth did not look rectangular.

Solution: I added a Black Outline (GL_LINE_LOOP) around every part. This made the gear look exactly like my pencil drawing on the graph paper.

Problem 2: The Coordinates were hard to find

It was difficult to guess where the corners of the teeth should be.

Solution: I used the graph paper method. I drew the gear physically and counted the grid squares. This gave me the exact numbers (0.2 width) to put in my code.

Problem 3: Flickering Screen

When I spun the gear fast, the image flashed on and off.

Solution: I changed the display mode from GLUT_SINGLE to GLUT_DOUBLE. This smoothed out the animation completely.

8.0 CONCLUSION

In conclusion, I successfully completed Assignment 1. I created a 2D mechanical gear that looks like my manual design. I met all the requirements of the lecturer.

I learned that:

1. **Planning is key:** Drawing on graph paper first makes coding much easier.
2. **Order matters:** In OpenGL, the order of Translation and Rotation changes the result.
3. **Primitives are powerful:** Complex objects like gears are just made of simple triangles and squares put together.

This assignment has improved my understanding of C++ and Computer Graphics significantly.